ETAPA C · DEMOSTRAR · SPRINT 16 DE 17

Sprint 16 · Reto «Entrega final: evaluar, demostrar y documentar» · Entrega y rúbrica

TABLA A · LOS FICHEROS

qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el programa) y **cuáles se suben y adónde**.

TABLA B · LOS REQUISITOS

qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

EN TOTAL, AL FINAL DEL SPRINT SE SUBEN A `SPRINT-16/`

9 ficheros del equipo (`README.md` , `notebooks/04_evaluacion.ipynb` , `demostrador/logica.py` , `demostrador/app.py` , `demostrador/tests/test_logica.py` , `demostrador/requirements.txt` , `docs/riesgos_eticos.md` , `docs/model_card.md` , `docs/memoria_tecnica.pdf`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **9 + tantos ficheros como personas haya en el equipo** (un equipo de 4 sube 13). «El reto» no es un fichero aparte: es ese conjunto, y las carpetas `notebooks/` , `demostrador/` y `docs/` van **dentro de** **`sprint-16/`** . Además: el vídeo plan B va a `docs/` solo si pesa menos de 25 MB (si no, su enlace va en el README); el modelo con su `metadatos.json` y los datos solo si pesan poco (si no, el README explica cómo generarlos o descargarlos); y la coevaluación confidencial (anexo B) **no se sube al repositorio**: se entrega al docente. Lo que ya subisteis en los sprints 14 y 15 (datos, código, registro de experimentos) no hay que subirlo otra vez. La *review* y la *retro* se hacen de palabra: no se suben.

Qué se sube y cómo se evalúa. Se lee **una vez**, antes de entregar, y se vuelve a la rúbrica cuando el equipo revisa su trabajo. Mientras trabajáis no hace falta: para eso está el reto en una página.

- Volver al README del sprint · ficha del reto, para consultar.
- Vuestra hoja personal: autoevaluación y coevaluación.
- Cómo subir a GitHub paso a paso, con capturas: Cómo entregar en GitHub. Esta hoja responde a **dos preguntas distintas**, una tabla para cada una:

1 · Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-16/`** , salvo los datos.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`README.md`	**Lo modificáis.** Partid del `README.md` de la raíz (que salió de la plantilla del Sprint 14); la lista de apartados está en el bloque A de la lista de congelación. Subid la versión final a `sprint-16/` y dejad también la raíz al día	Escribirlo: problema, resultados, estructura de carpetas, cómo reproducir, cómo lanzar la demo, datos y licencias, IA responsable y equipo. Al final, la sección «Revisión con la rúbrica»	**Sí** · `sprint-16/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`notebooks/04_evaluacion.ipynb`	**Lo creáis a partir de un cuaderno.** Os damos `S13_02` con los gráficos	Adaptarlo a vuestro modelo: evaluación en test, gráficos, análisis de errores, rendimiento por subgrupos y mitigación medida	**Sí** · `sprint-16/notebooks/`
`demostrador/logica.py`	**Lo modificáis.** Plantilla con validación, nivel de riesgo y aviso de incertidumbre	Cambiar los campos, las reglas de validación y el texto del aviso por los de vuestro problema	**Sí** · `sprint-16/demostrador/`
`demostrador/app.py`	**Lo modificáis.** Plantilla de interfaz con Streamlit	Adaptar el formulario a vuestros campos. Si usáis Gradio u otra herramienta, el fichero se llama distinto pero cuenta igual	**Sí** · `sprint-16/demostrador/`
`demostrador/tests/test_logica.py`	**Lo modificáis.** Pruebas de sentido común para `pytest`	Cambiar la reserva de ejemplo por una de vuestro proyecto y añadir tres pruebas propias	**Sí** · `sprint-16/demostrador/tests/`
`demostrador/requirements.txt`	**Lo modificáis.** Trae las versiones del entorno de la plantilla	Recortarlo a lo que usáis y añadir lo que use el cuaderno de evaluación. Indicad la versión de Python en el README	**Sí** · `sprint-16/demostrador/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`docs/riesgos_eticos.md`	**Lo modificáis.** Es el registro de riesgos que empezasteis en el Sprint 14 y revisasteis en el 15	Cerrarlo: cada riesgo alto con su estado y su evidencia, y la clasificación RGPD / AI Act justificada	**Sí** · `sprint-16/docs/`
`docs/model_card.md`	**Lo creáis a partir de una plantilla.** `plantilla_model_card.md`	Rellenar sus 9 apartados con las cifras finales. Máximo 2 páginas	**Sí** · `sprint-16/docs/`
`docs/memoria_tecnica.pdf`	**Lo creáis a partir de una plantilla.** `plantilla_esqueleto_memoria_tecnica.md`	Escribir el texto en el programa que uséis y exportarlo a PDF. 10–15 páginas sin contar anexos	**Sí** · `sprint-16/docs/`
`docs/demo.mp4` (vídeo plan B)	**Lo creáis.** Grabación de pantalla	Grabar 1–2 minutos del demostrador funcionando	Solo si pesa menos de 25 MB · `sprint-16/docs/` ; si no, su enlace en el README
`demostrador/modelo/` (modelo y `metadatos.json`)	**Lo genera** el cuaderno `S16_01` o vuestro script de entrenamiento	Ejecutarlo; no se escribe a mano	Solo si pesan poco; si no, el README explica cómo generarlos · `sprint-16/demostrador/`
`demostrador/api.py` (extra)	**Lo modificáis.** Plantilla de API con FastAPI	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-16/demostrador/`
Datos del proyecto o script de descarga	**Ya hecho** en el Sprint 14	Nada: comprobad que el README dice de dónde salen y con qué licencia	Solo si faltan y pesan poco · `data/`
`checklist_congelacion.md`	**Ya hecha, en blanco**	Pasarla entera con una persona que no escribió el README	**No** se sube: se rellena en papel o en el tablero

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
Coevaluación individual (anexo B)	**Ya hecha, en blanco**	Cada persona rellena la suya	**No** se sube al repositorio: se entrega al docente
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación de la *review* va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-16/`

2 · Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Evaluación final en el test	Se evalúa **una sola vez** el modelo candidato del Sprint 15, con las métricas y su incertidumbre, y gráficos adaptados de `S13_02` (matriz de confusión, curvas, importancia de variables), cada uno con un título que dice la conclusión	Mínimo	`notebooks/04_evaluacion.ipynb`
Rendimiento por subgrupos	La misma métrica para cada subgrupo relevante (canal, país, tipo de imagen…). Si hay diferencias grandes, se anota el sesgo	Mínimo	Cuaderno y apartado 6 de la model card
Análisis de errores	Lista de los casos donde más falla, con ejemplos reales y lo que tienen en común	Estándar	Cuaderno y memoria (sección 5)
Mitigación del sesgo	Una medida elegida y aplicada: reponderar, ajustar el umbral por grupo, añadir datos del grupo que falta o limitar el uso. **El umbral o los pesos se eligen con validación cruzada sobre el entrenamiento**; el test solo mide el antes y el después	Mínimo	Cuaderno y memoria (sección 6)
Mitigación medida	Métricas **antes y después, por subgrupo**: qué mejora, qué empeora y qué cuesta (por ejemplo, más falsas alarmas). Si no se puede arreglar, se documenta como limitación	Estándar	Cuaderno, model card y memoria (sección 6)
Riesgos y normativa	Registro cerrado: cada riesgo alto con estado y evidencia. Clasificación RGPD / AI Act justificada, con supervisión humana y transparencia	Mínimo	`docs/riesgos_eticos.md` y model card
Demostrador	Se lanza con **un comando** indicado en el README y funciona desde cero en otro equipo. No guarda datos personales ni lleva claves	Mínimo	`demostrador/` y `README.md`

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Entradas erróneas e incertidumbre	Se prueba con 3 entradas normales, 1 imposible y 1 vacía: mensajes claros y sin romperse. Cada respuesta lleva un aviso de incertidumbre (por ejemplo, «Estimación orientativa. El modelo tiene un AUC de 0,75 en datos de prueba…»)	Estándar	`demostrador/logica.py` y `app.py`
Pruebas automáticas	`pytest -q` pasa, con al menos tres pruebas propias de sentido común	Estándar	`demostrador/tests/test_logica.py`
Plan B en vídeo	Vídeo de 1–2 minutos (unos 90 segundos) del demostrador funcionando	Estándar	`docs/demo.mp4` o enlace en el README
README y versiones	README con los apartados indicados y `requirements.txt` con **versiones fijas** y versión de Python	Mínimo	`README.md` y `demostrador/requirements.txt`
Prueba en limpio	Una persona que **no escribió el README** lo sigue en un Colab nuevo o una carpeta limpia (*Code → Download ZIP*) y lo anota. Se corrige lo que falle	Estándar	`README.md` (sección «Revisión con la rúbrica»)
Model card	Las 9 partes de la plantilla, máximo 2 páginas. Coincide con la memoria y con el aviso del demostrador	Mínimo	`docs/model_card.md`
Memoria técnica	PDF de 10–15 páginas sin contar anexos, propia y autocontenida, con las secciones del esqueleto, figuras con pie y referencias. Si usáis IA para redactar, se revisa y se declara	Mínimo	`docs/memoria_tecnica.pdf`
Cifras coherentes y conclusiones accionables	Las mismas cifras en memoria, README, model card y registro de experimentos. La memoria dice qué recomendáis al cliente y qué **no** debe hacerse con el modelo	Estándar	`docs/memoria_tecnica.pdf` (sección 9)
Git y congelación	Commits de **todas** las personas. Etiqueta `v1.0-entrega` creada **antes de las 14:30 del 10/12**. Ni claves ni datos personales. Nada se cambia después	Mínimo	Historial y etiquetas del repositorio
Lista de congelación	Los bloques A a F pasados enteros, con una persona que no escribió el README	Estándar	`checklist_congelacion.md` (en papel o en el tablero)
Coevaluación individual (anexo B)	Cada integrante entrega la suya al docente. Junto con los commits, ajusta la nota individual (de 0,8 a 1,1)	Mínimo	Entrega al docente
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa el repositorio y la memoria con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el README; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`README.md` (sección «Revisión con la rúbrica») y `sprint-16/`
Extras	SHAP o Grad-CAM interpretado con palabras · métricas con intervalo de incertidumbre · comparar dos medidas de mitigación · comprobar el rendimiento con datos distintos (otro periodo, fotos reales…) · `api.py` con `/salud` y `/predecir`	Extra	Cuaderno · memoria · `demostrador/api.py`

3 · Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de congelar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**. Esta rúbrica sirve para la nota del sprint; la nota del **proyecto** usa la rúbrica de 100 puntos del documento de briefs.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay evaluación, pero el test se ha usado más de una vez o solo hay una métrica global; el demostrador no arranca	Evaluación en test hecha una vez con métricas y gráficos; métrica por subgrupos; sesgo anotado y medida elegida con el entrenamiento (no con el test) y aplicada; el demostrador arranca en el equipo de quien lo hizo	Además, análisis de errores con ejemplos reales; mitigación medida antes y después por subgrupo; el demostrador rechaza entradas erróneas con un mensaje claro y `pytest -q` pasa	Además, el modelo se interpreta con SHAP o Grad-CAM, hay intervalos de incertidumbre, se comparan dos mitigaciones o se prueba con datos distintos
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona en el ordenador de quien lo hizo; el README no dice cómo ejecutarlo	El README tiene los comandos y hay `requirements.txt`, pero faltan versiones, datos o la versión de Python	Otra persona, en un Colab nuevo o una carpeta limpia, lo ejecuta **siguiendo solo el README** y ve el demostrador; versiones fijas; commits de todas las personas; etiqueta `v1.0-entrega` creada antes de las 14:30	Además, el README indica el tiempo total de reproducción, la estructura de carpetas y un comando único para la demo, y la prueba en limpio está anotada (quién, qué falló, cómo se corrigió)
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	La memoria está incompleta o la model card tiene huecos esenciales	Memoria de 10–15 páginas con todas las secciones y model card completa, pero con conclusiones genéricas o cifras que no coinciden	Memoria clara y autocontenida, con figuras con pie y referencias; cifras iguales en todos los documentos; conclusiones accionables (qué hacer y qué **no** hacer con el modelo); el demostrador comunica la incertidumbre	Además, un resumen ejecutivo que se entiende sin leer el resto y recomendaciones priorizadas con lo que habría que hacer antes de un despliegue real
IA responsable (¿se sabe qué riesgos hay y qué se hizo?)	El registro de riesgos no está cerrado o no hay análisis por subgrupos	Registro con estado de cada riesgo alto, clasificación RGPD / AI Act citada, sesgo detectado y aviso de límites en el demostrador	Clasificación **justificada**; mitigación aplicada **y medida**; limitaciones y usos no permitidos en la model card; el uso de IA declarado (qué se pidió, qué se revisó y qué se cambió); sin datos personales ni claves	Además, supervisión humana y transparencia explicadas, y se cuenta un fallo concreto (de la IA o de la mitigación, por ejemplo, un subgrupo que empeoró) y qué se decidió

Cómo se revisa (unos 20 minutos, antes de crear la etiqueta `v1.0-entrega`, que se crea justo después de la revisión; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar**, en una carpeta limpia o en un Colab nuevo, lo que dice el README: instalar, abrir el cuaderno de evaluación, lanzar el demostrador y ejecutar `pytest -q`, copiando solo lo que pone ahí. Anotar lo que falla y cuánto se tarda.
2. **Probar el demostrador** con 3 entradas normales, 1 imposible y 1 vacía: ¿mensajes claros? ¿avisa de la incertidumbre?

3. **Leer la memoria y la model card como si fuerais el cliente**: ¿se entiende qué recomendáis y cuándo no fiarse del modelo? Comprobar que las cifras son iguales en todos los documentos y que el registro de riesgos está cerrado.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · el README funciona en un Colab nuevo, pero falta la versión de Python»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `README.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.